Техническое задание SXSVN

Актуальное распределение: **Данил — парсинг и подготовка данных; Александр (Санёк) — модель и AI-агент; Владислав — сервер, API, интерфейс и интеграция.** Подробные задачи и критерии готовности — в разделе 7.

1. Общая архитектура

SXSVN — система персональной бизнес-аналитики с двумя основными контурами:

1. **Подготовка новостного потока** — сбор, очистка, дедупликация, извлечение фактов и предварительная классификация событий.

2. **Анализ влияния на компанию** — использование профиля компании, платежных связей, новостей и аналитики Альфа-Банка для формирования проверяемых бизнес-инсайтов.

Общий поток данных:

```
Источники данных
  ├─ Новости и внешние источники
  ├─ CSV компаний
  ├─ Платежи и связи контрагентов
  └─ Аналитические материалы Альфа-Банка
          |
          v
A. Конвейер подготовки данных
  Импорт → очистка → нормализация → дедупликация → кластеризация
          |
          v
B. Индексы и профиль компании
  Профили компаний · платежные признаки · индекс событий · индекс документов банка
          |
          v
C. Отбор событий
  Программные фильтры → LLM-классификация → персональный приоритет
          |
          v
D. AI-агент
  Планирование → инструменты → анализ → проверка → адаптация
          |
          v
E. API и интерфейс
  Карточки событий · источники · риски · возможности · уточняющие вопросы
```

Основной принцип: каждый блок имеет определенные входы, выходы и критерии успешного выполнения. Не следует реализовывать всю систему одним большим промптом.

2. Контур A сбор, очистка и подготовка новостей

Ответственный: Данил. Александр предоставляет функции LLM-извлечения фактов и классификации; Данил подключает их к конвейеру. Владислав обеспечивает серверный запуск и сохранение результатов.

Задача контура — преобразовать поток публикаций в каталог уникальных событий.

Шаг 1. Получение новых материалов

Использовать доступные и разрешенные источники: RSS, API и другие предусмотренные проектом каналы.

Для каждого материала сохранять:

• исходный URL и источник;

• заголовок;

• время публикации и время получения;

• полный доступный текст;

• идентификатор материала;

• статус загрузки и ошибки.

Сохранять материал в хранилище сразу после получения, чтобы не загружать и не обрабатывать всё заново при сбое.

Шаг 2. Очистка текста

До обращения к LLM обычный код должен:

1. Удалять HTML-теги, меню, рекламу и навигацию.

2. Нормализовать пробелы, переносы строк и кодировку.

3. Убирать повторяющиеся блоки и технические подписи.

4. Хранить заголовок, лид, основной текст и метаданные отдельно.

5. Проверять минимальную длину текста и наличие полезного содержимого.

Не удалять автоматически все числа, даты, названия организаций или короткие абзацы: они могут быть важны для финансового анализа.

Шаг 3. Дедупликация

Использовать несколько уровней:

• одинаковый нормализованный URL — вероятный дубль;

• одинаковый хеш очищенного текста — точный дубль;

• похожие заголовки и тексты — кандидат на дубль или повторную публикацию;

• совпадающее событие в разных СМИ — кластер публикаций об одном событии.

Две статьи об одном событии не всегда взаимозаменяемы. Более поздняя публикация может содержать новые цифры или официальное опровержение. Поэтому в кластере следует сохранять все источники и отмечать, какие факты добавляет каждый из них.

Шаг 4. Извлечение фактов

После очистки LLM получает новый, еще не обработанный материал и возвращает структурированные данные. Пример схемы:

```
{
  "event_id": "event_001",
  "topic": "regulation",
  "event_date": null,
  "published_at": "2026-10-20T10:00:00+03:00",
  "actors": [],
  "facts": [],
  "affected_sectors": [],
  "geographies": [],
  "source_ids": [],
  "unverified_details": []
}
```

Это демонстрационная схема, а не содержимое реальной новости.

Каждый факт должен ссылаться на исходный фрагмент текста. Если дата события не установлена, оставить null, а не подставлять дату публикации.

Шаг 5. Предварительный отбор

Сначала применяются программные правила: тематика, география, актуальность, дубликаты и очевидно нерелевантные категории. Затем LLM-классификатор определяет одно из решений:

• KEEP — передать на дальнейший анализ;

• DROP — отсеять как нерелевантное;

• REVIEW — проверить дополнительно из-за неоднозначности или потенциальной важности.

Не следует отбрасывать событие только потому, что заголовок не совпадает с отраслью компании: влияние может быть косвенным, через контрагентов.

Результат контура A: каталог событий с первоисточниками и извлеченными фактами, а не готовые персональные инсайты.

3. Контур B подготовка данных о компаниях

Этот блок реализует Данил. Владислав подключает подготовленные данные к серверному хранилищу и API. Качество данных здесь непосредственно влияет на точность AI-агента.

Шаг 6. Импорт CSV и аудит данных

До реализации бизнес-логики проверить:

• кодировку, разделители и типы столбцов;

• уникальность идентификаторов компаний;

• единицы измерения выручки и сумм платежей;

• пропуски и некорректные значения;

• направление платежей;

• долю платежных связей, для которых удалось найти обе компании;

• наличие компаний нужного масштаба.

Не следует придумывать период платежей, если он не указан в данных.

Шаг 7. Создание профиля компании

Для каждой компании сформировать профиль, который быстро извлекается по company_id.

Пример:

```
{
  "company_id": "COMPANY_001",
  "industry": {
    "value": "manufacturing",
    "source": "company_csv"
  },
  "revenue": {
    "value": null,
    "unit": null
  },
  "bank_products": [],
  "payment_exposure": {},
  "known_counterparties": [],
  "coverage": {},
  "missing_data": []
}
```

В реальной реализации нужно сохранять происхождение каждого существенного признака: файл, строку или расчет, из которого он получен.

Не передавать LLM все CSV целиком. Вычислять агрегаты обычным Python-кодом и передавать только профиль конкретной компании и признаки, релевантные событию.

Шаг 8. Расчет связей и экспозиции

Для MVP ограничиться одним шагом графа: компания → непосредственный контрагент.

Алгоритм:

1. Определить затронутую отрасль или категорию контрагентов.

2. Найти связанные с компанией организации.

3. Проверить, какие из них соответствуют условиям события.

4. Рассчитать наблюдаемые платежные показатели.

5. Рассчитать покрытие известных контрагентов.

6. Передать результаты агенту вместе с ограничениями.

Пример показателя для анализа входящих платежей:

E = сумма платежей от затронутых контрагентов / сумма всех валидных входящих платежей.

Формула подходит, если релевантен именно канал входящих платежей. Для исходящих платежей нужен отдельный расчет.

Если знаменатель равен нулю, результат неизвестен. Если часть контрагентов не идентифицирована, их нельзя автоматически считать незатронутыми.

E — доля наблюдаемых платежей, а не вероятность убытка, доля выручки или доказанная зависимость от поставщика.

4. Контур C отбор событий для конкретной компании

Ответственные: Данил готовит признаки и программный отбор кандидатов; Александр определяет и реализует модельную оценку релевантности и итогового приоритета. Владислав подключает запуск через API.

Шаг 9. Программное ранжирование

Для каждой пары «компания — событие» рассчитывать предварительный приоритет по доступным признакам:

• соответствие отрасли;

• соответствие банковским продуктам;

• релевантность контрагентов;

• географическая связь;

• актуальность события;

• масштаб наблюдаемого воздействия.

Разделять два показателя:

• priority_score — насколько важно обратить внимание на событие;

• evidence_status — насколько хорошо подтверждены основания анализа.

Высокий приоритет при слабых доказательствах означает, что событие стоит проверить, но категоричный вывод делать нельзя.

Шаг 10. LLM-классификация

Для событий, прошедших предварительные фильтры, модель отвечает на ограниченный вопрос:

Есть ли основания считать событие потенциально значимым для этой компании? Какие конкретные признаки это подтверждают и каких данных не хватает?

На вход подавать краткую карточку события и профиль компании, а не всю статью и всю базу.

Возможные решения:

• KEEP — событие стоит анализировать подробнее;

• DROP — нет достаточных оснований считать его релевантным;

• REVIEW — нужно проверить дополнительные данные.

Потенциально критические события могут попадать в углубленный анализ по отдельным правилам независимо от классификатора.

Шаг 11. Подготовка контекста для агента

Для каждого события со статусом KEEP или REVIEW создавать пакет:

• факты события и первоисточники;

• профиль компании;

• релевантные признаки и связи;

• расчеты экспозиции;

• результаты предварительного ранжирования;

• недостающие данные;

• найденные документы банка.

Это единый контекст для insight_engine.

5. Контур D логика AI-агента

Это основная зона ответственности Александра (Санька). Агент управляет анализом, вызывает инструменты, проверяет результат и при необходимости адаптирует ход работы.

Шаг 12. Создание состояния задачи

В начале каждого анализа агент создает объект состояния. Это не вся история переписки, а компактный набор данных, необходимых для текущей задачи.

```
state = {
    "company_id": "COMPANY_001",
    "event_id": "event_001",
    "as_of_date": "2026-10-20",
    "profile": {},
    "event_facts": [],
    "bank_evidence": [],
    "calculations": [],
    "claims": [],
    "missing_data": [],
    "tool_trace": [],
    "retry_count": 0,
    "status": "initialized"
}
```

По мере выполнения шагов агент обновляет состояние. При повторном поиске он использует уже собранные факты, а не начинает анализ заново.

Шаг 13. Планирование анализа

LLM получает краткий профиль компании и факты события, после чего предлагает возможные каналы влияния.

План должен содержать:

• предполагаемый механизм влияния;

• признаки, необходимые для проверки;

• требуемые источники;

• инструменты, которые нужно вызвать;

• неизвестные параметры;

• условия завершения анализа.

На этом этапе это гипотезы, а не итоговые выводы.

Шаг 14. Вызов инструментов

Граница ответственности: Данил реализует функции получения профилей, расчётов и поиска по подготовленным данным. Александр описывает инструменты для агента, управляет их вызовами и проверяет смысловую обоснованность выводов. Владислав подключает функции к хранилищу, обеспечивает права доступа, статусы задач и обработку серверных ошибок.

Рекомендуемый набор инструментов:

Инструмент	Вход	Результат
get_company_profile	ID компании	Профиль и неизвестные признаки
get_counterparty_exposure	ID компании, тема события	Связанные организации, показатели и покрытие
search_news	Запрос, период, фильтры	Материалы с ID, датами и ссылками
search_bank_reports	Тема и контекст компании	Фрагменты аналитики банка с датами и страницами
calculate_exposure	Проверенные данные и параметры расчета	Число, формула, единицы и входные ID
validate_evidence	Утверждения и доказательства	Ошибки и неподтвержденные утверждения

Каждый инструмент должен возвращать не только данные, но и статус выполнения: success, empty, partial или error.

Так агент отличит ситуацию, когда источник ничего не нашел, от ситуации, когда источник недоступен.

Шаг 15. Поиск аналитики Альфа-Банка

Данил: извлечение текста, метаданных и страниц, разбиение на фрагменты и базовый поисковый индекс. **Александр:** формирование поисковых запросов, выбор релевантных фрагментов и проверка применимости к выводу. **Владислав:** хранение, доступ к документам из интерфейса и серверное подключение поиска.

Для MVP подготовить индекс документов банка вместо загрузки всех отчетов в каждый запрос.

1. Извлечь текст документов.

2. Сохранить название, дату, страницы и фрагменты.

3. Разбить документы на смысловые части.

4. Искать по теме события и нужному каналу воздействия.

5. Проверить дату и релевантность найденного материала.

6. Передать найденные фрагменты в контекст агента.

Если подходящего документа нет, результат должен явно сообщать об этом. Общий ответ LLM нельзя выдавать за позицию Альфа-Банка.

Шаг 16. Формирование причинно-следственных гипотез

Каждый вывод строится по шаблону:

Факт события → признак компании → механизм воздействия → возможное последствие → ограничение.

Например:

• Факт: изменились условия работы в определенной отрасли.

• Признак: часть входящих платежей компании связана с контрагентами из этой отрасли.

• Механизм: изменения могут повлиять на деятельность этих контрагентов.

• Возможное последствие: может измениться объем или срок платежей.

• Ограничение: нет подтверждения, что конкретные контрагенты действительно столкнулись с проблемой.

Такой вывод может обосновать дополнительную проверку, но не утверждение о том, что компания обязательно потеряет выручку.

Шаг 17. Проверка результата

Программные проверки:

• Все ID источников существуют.

• Ссылки относятся к нужным материалам.

• Числа совпадают с расчетами.

• Даты и единицы измерения корректны.

• JSON соответствует схеме.

• Использованы только разрешенные инструменты.

Смысловые проверки:

• Есть ли связь между событием и компанией?

• Подтвержден ли каждый существенный факт?

• Не представлена ли гипотеза как доказанный факт?

• Учитывает ли вывод ограничения данных?

• Не противоречат ли друг другу источники?

• Действительно ли аналитика банка относится к теме?

Повторная генерация той же моделью может выявить некоторые проблемы, но не является доказательством истинности ответа. Для смысловой проверки нужны подготовленные тестовые случаи и независимая оценка команды.

Шаг 18. Решение о следующем действии

После проверки агент выбирает один из четырех путей:

1. **Завершить.** Данных достаточно, существенные утверждения обоснованы.

2. **Повторить поиск.** Не хватает конкретного факта, который можно найти в доступных источниках.

3. **Спросить пользователя.** Недостающий факт способен существенно изменить вывод и пользователь может его уточнить.

4. **Сузить вывод.** Дополнительные данные недоступны; вернуть ограниченный анализ с явными оговорками.

Установить максимум два дополнительных шага поиска за один анализ. Если проблема остается, агент не должен бесконечно повторять запросы или имитировать успешную проверку.

6. Контур E формирование ответа пользователю

Александр формирует структурированный ответ и обновляет анализ после уточнения. **Владислав** реализует API, интерфейс и сохранение уточнений. **Данил** обеспечивает корректность исходных данных и повторных расчётов.

Шаг 19. Сборка итогового инсайта

После завершения анализа агент формирует структурированный результат. Пример:

```
{
  "event": {
    "event_id": "event_001",
    "title": "Изменение условий в отрасли",
    "event_date": null,
    "published_at": "2026-10-20"
  },
  "priority": {
    "score": 0.82,
    "reasons": [
      "Обнаружена связь с контрагентами"
    ]
  },
  "company_features": [],
  "bank_view": [],
  "ai_implications": [],
  "risks": [],
  "opportunities": [],
  "missing_data": [],
  "next_check": {},
  "evidence": [],
  "trace_id": "trace_001",
  "status": "partial"
}
```

Это иллюстрация контракта. Число 0.82 условное, а partial означает неполный результат. В настоящем приложении статус и приоритет должны вычисляться по определенным правилам.

Каждое утверждение в risks, opportunities и ai_implications должно содержать ссылки на доказательства и ограничения. Приоритет не должен интерпретироваться как вероятность убытка.

Шаг 20. Отображение в интерфейсе

Интерфейс должен показывать:

• **Событие:** что произошло, когда, где опубликовано и ссылка на первоисточник.

• **Почему это важно компании:** конкретный признак компании или измеренная связь с контрагентами.

• **Возможные последствия:** риски, возможности, механизм влияния и условия, при которых вывод может измениться.

• **Основания и позиция банка:** факты, документы Альфа-Банка, даты, страницы, предположения AI и недостающие данные.

• **Что проверить дальше:** один конкретный вопрос или действие, которое способно уточнить анализ.

Шаг 21. Уточнение пользователя

Если пользователь отвечает на вопрос агента, не нужно просто добавлять ответ в текст диалога и заново генерировать всё с нуля.

1. Сохранить уточнение с привязкой к событию и компании.

2. Определить, какое предположение или неизвестное поле оно уточняет.

3. Обновить соответствующий признак состояния.

4. Повторно рассчитать зависимые показатели.

5. Пересмотреть только затронутые гипотезы.

6. Повторить проверку и вернуть обновленную карточку с новым trace_id или версией результата.

Например, подтверждение отсутствия отсрочки платежа у контрагента может снизить значимость гипотезы о задержке дебиторской задолженности, но не обязательно влияет на другие каналы воздействия.

7. Распределение ответственности в команде

Данил парсинг и подготовка данных

Задача: превращать исходные новости, PDF и CSV в проверенные данные, пригодные для анализа моделью.

1. Подключить источники новостей, сохранять исходные тексты, URL, даты и статусы загрузки.

2. Реализовать очистку, нормализацию, точные дубли и группировку публикаций об одном событии. Сохранять новые факты и опровержения.

3. Подготовить CSV компаний, платежей и связей: типы, единицы, пропуски, ID, направления платежей и покрытие соединений.

4. Реализовать профиль компании, связи на один шаг, расчёты входящих и исходящих платежных долей с ограничениями данных.

5. Извлечь текст аналитики банка по страницам, определить метаданные и подготовить поисковый индекс. Не выдавать дату загрузки за дату отчёта.

6. Сделать функции get_company_profile, get_counterparty_exposure, calculate_exposure, search_news и базовый search_bank_reports. Владислав подключает их к серверу, Александр — к агенту.

7. Подготовить широкий программный отбор кандидатов; подключить предоставленные Александром функции извлечения фактов. Не строить несколько независимых LLM-фильтров.

8. Передать компактный проверочный набор и отчёт о качестве данных; указать неизвестные параметры и происхождение признаков.

Передаёт команде: модуль подготовки, согласованные JSON-примеры, каталог источников и событий, профили, индекс отчётов, функции расчётов и инструкция обновления.

Готово, когда: новая публикация проходит обработку без ручного редактирования; повторный запуск не создаёт дублей; выбранная компания получает воспроизводимый профиль; все расчёты имеют входные ID, единицы и формулу; фрагменты PDF сохраняют страницы.

Александр (Санёк) модель и AI-агент

Задача: на подготовленных данных формировать обоснованный персональный разбор и адаптировать анализ при нехватке информации.

1. Подключить разрешённую LLM через выданный API, проверить доступность, учитывать токены, стоимость и лимиты.

2. Определить схемы извлечённых фактов, итогового ответа, доказательств и состояния агента; согласовать их с Данилом и Владиславом.

3. Реализовать модельное извлечение фактов и оценку релевантности. Извлечение выполнять один раз для новой версии материала, а не для каждой компании.

4. Разработать insight_engine: планирование, выбор инструментов, использование профиля, связей и аналитики банка, генерация рисков и возможностей.

5. Реализовать проверку утверждений, разделение фактов, позиции банка и гипотез AI; повторный поиск, вопрос пользователю либо сужение вывода.

6. Возвращать структурированный ответ, статус и журнал действий; сохранять ссылки от утверждений к конкретным фрагментам источников.

7. Подготовить проверки прямого и косвенного влияния, нерелевантности и неполных данных; измерить качество, задержку и расходы.

Передаёт команде: вызываемый модуль анализа, модельные функции для конвейера Данила, схемы и примеры ответов, тестовые случаи и результаты замеров. Обучение собственной модели не является обязательной задачей.

Готово, когда: один запрос даёт валидный разбор с основаниями; агент действительно меняет действия при пробеле; уточнение пользователя меняет соответствующий вывод; журнал позволяет восстановить источники и инструменты; неподтверждённое не выдаётся за факт.

Владислав сервер и сборка работающего приложения

Задача: соединить подготовку Данила и анализ Александра в приложение, которым можно пользоваться и которое жюри сможет запустить.

1. Создать сервер приложения и схему хранения компаний, материалов, событий, результатов, задач и уточнений. Согласовать структуры с командой до интеграции.

2. Подключить модуль подготовки Данила: импорт подготовленных данных, запуск обновления, сохранение статусов и ошибок. Не дублировать парсеры и формулы расчётов.

3. Подключить insight_engine Александра: запуск анализа, передача контекста, сохранение результата и журнала, повторный анализ после уточнения.

4. Реализовать API: список компаний, лента событий, запуск анализа, статус задачи, получение карточки и источников, отправка ответа на вопрос агента. Названия маршрутов закрепить в общем контракте.

5. Сделать интерфейс: главная страница с постоянным входом в сервис, выбор компании, карточки, причины приоритета, риски и возможности, позиция банка, источники и уточняющий вопрос.

6. Показывать состояния «загрузка», «готово», «частичный результат», «нет данных» и «ошибка». Долгий запрос модели не должен оставлять пользователя без статуса.

7. Реализовать серверную проверку доступа к демо-компаниям, хранение ключей в окружении, таймауты, журналирование и безопасную обработку входных данных. Ключ модели не передавать браузеру.

8. Собрать Docker, зафиксировать зависимости, написать README, .env.example без секретов и пошаговый сценарий проверки. При наличии времени разместить приложение на внешнем сервере.

9. Провести интеграционную проверку на чистом окружении и подготовить комплект сдачи: версия Git, доступы для жюри, ссылки, инструкция и презентация. Технические сведения и результаты для презентации предоставляют Данил и Александр.

Передаёт команде: работающее приложение, описание API, интерфейс, конфигурацию запуска, инструкцию и комплект сдачи.

Готово, когда: другой участник запускает проект по README, выбирает компанию, получает анализ с источниками, отвечает на вопрос и видит обновление; сбои понятны; секреты не попадают в репозиторий или интерфейс.

Общие границы и проверка

Компонент	Основной владелец	Кто подключает или проверяет
Сбор, очистка, дубли новостей	Данил	Владислав подключает запуск и хранение
CSV, профили, связи и расчёты	Данил	Александр проверяет применение в выводах
PDF, метаданные, фрагменты и поисковый индекс	Данил	Александр задаёт поиск; Владислав подключает хранение
LLM-извлечение фактов	Александр	Данил подключает в конвейер
Релевантность, промпты, агент и смысловая проверка	Александр	Данил проверяет числа и основания
API, база и интеграция модулей	Владислав	Данил и Александр предоставляют функции и схемы
Интерфейс и уточнения пользователя	Владислав	Александр обновляет анализ; Данил обеспечивает расчёты
Docker, README и комплект сдачи	Владислав	Все проверяют запуск и содержание

Ключевое правило: **сначала согласовать контракт между модулями, затем разрабатывать параллельно**. Данил не меняет company_id, Александр — структуру evidence, Владислав — API без согласования с теми, кого изменение затрагивает.

Первый общий результат: Данил передаёт профиль, новость и фрагмент отчёта; Александр возвращает один проверяемый разбор; Владислав показывает его в интерфейсе. Затем добавляются автоматический поиск, связи и адаптация.

8. Последовательность реализации

Этап 1. Контракты и доступ к данным

Владельцы: Данил — примеры данных; Александр — API модели и схема ответа; Владислав — контракт API приложения и каркас запуска.

• Зафиксировать JSON-схемы.

• Проверить API разрешенной LLM.

• Получить один профиль компании и одну тестовую новость.

Критерий готовности: участники могут обмениваться данными через согласованный формат.

Этап 2. Рабочий конвейер новостей

Владельцы: Данил — парсинг, CSV и подготовка отчётов; Александр — функция извлечения фактов; Владислав — хранение и подключение конвейера.

• Реализовать импорт, очистку, дедупликацию и извлечение фактов.

• Сохранять источники.

• Параллельно проверить CSV и подготовить профиль компании.

Критерий готовности: новую статью можно обработать без ручной подготовки.

Этап 3. Первый полноценный инсайт

Владельцы: Александр — анализ и проверка; Данил — достоверные входные данные; Владислав — API и отображение.

• Передать LLM факты, профиль компании и банковский материал.

• Сформировать вывод с доказательствами, рисками, возможностями и ограничениями.

• Провалидировать результат и отобразить его в интерфейсе.

Критерий готовности: результат проходит валидацию и отображается в приложении.

Этап 4. Планирование и адаптация агента

Владельцы: Александр — логика агента; Данил — инструменты данных; Владислав — сохранение состояния, вопросы и обновление карточки.

• Добавить выбор инструментов.

• Реализовать проверку достаточности данных.

• Добавить повторный поиск и уточняющие вопросы.

Критерий готовности: журнал показывает изменение последовательности действий в зависимости от ситуации.

Этап 5. Тесты и подготовка защиты

Владельцы: Данил — качество данных; Александр — качество выводов и стоимость; Владислав — сквозной запуск, упаковка и демонстрация. Финальную проверку проходят все.

• Проверить прямые и косвенные связи, противоречия и неполные данные.

• Проверить качество отбора и расход токенов.

• Сравнить результат с простой отраслевой лентой.

• Подготовить воспроизводимый запуск.

Критерий готовности: демонстрация воспроизводится, ограничения и ошибки отражаются честно.

9. Обязательные тестовые сценарии

1. **Прямое влияние:** новость напрямую касается отрасли компании.

2. **Косвенное влияние:** новость затрагивает отрасль контрагента.

3. **Нерелевантная новость:** система не создает искусственную связь.

4. **Недостаточно данных:** система запрашивает уточнение или ограничивает вывод.

5. **Противоречивые источники:** система обнаруживает расхождение и не скрывает его.

6. **Одна новость — две компании:** меняются основания, приоритет или содержание анализа.

7. **Повторная публикация:** дубли не приводят к повторным затратным вызовам LLM.

8. **Недоступный источник:** система различает пустой результат и техническую ошибку.

Отдельно измерять стоимость каждого запуска, количество вызовов модели, задержку, качество источников и долю важных событий, сохранившихся после фильтрации.

10. Ключевой принцип проекта

Собрать новости → очистить и объединить дубли → извлечь факты → подготовить профиль компании → отобрать релевантные события → собрать банковскую аналитику и деловые связи → проанализировать влияние → проверить доказательства → при необходимости уточнить данные → вернуть персонализированный инсайт.

Главная ответственность Александра — модель, анализ, проверка и адаптация. Данил отвечает за парсинг и подготовку проверяемых данных. Владислав — за сервер, интерфейс, интеграцию и воспроизводимый запуск приложения.

Самая важная техническая задача сейчас — определить состояние агента, контракты инструментов и правила переходов между шагами. Когда они зафиксированы, реализацию можно разделить на независимые модули, каждый из которых проще написать, проверить и интегрировать.